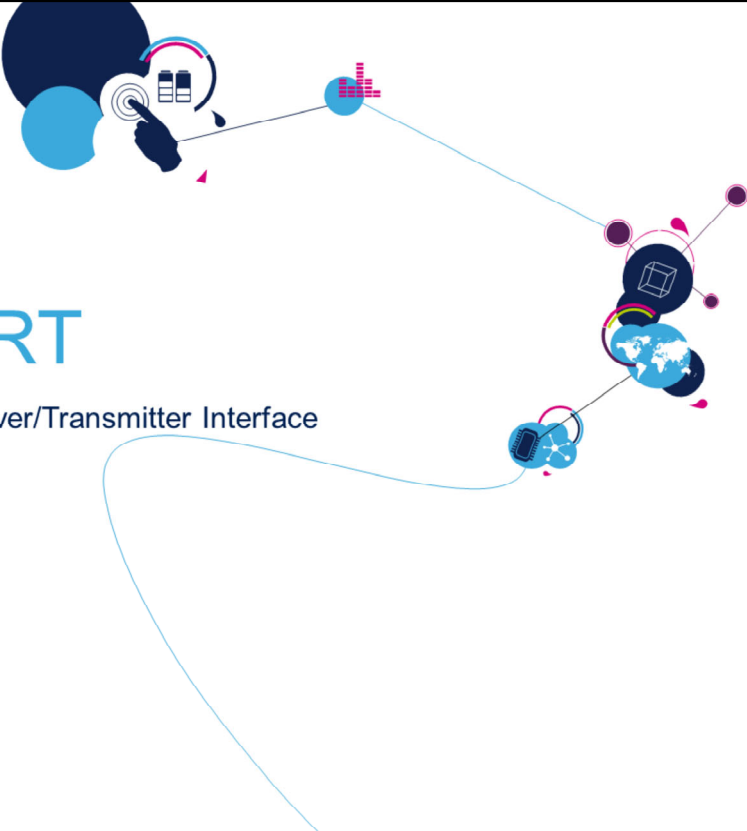




STM32G0 – USART

Universal Synchronous/Asynchronous Receiver/Transmitter Interface

Revision 1.0



Hello, and welcome to this presentation of the STM32 Universal Synchronous/Asynchronous Receiver/Transmitter Interface.

It covers the main features of this USART interface, which is widely used for serial communications in embedded systems.



Application benefits

- Multi-purpose communication peripheral
- Simple hardware, only a few pins needed
- Wakes from low-power mode
- Transmit and Receive FIFOs, with capability to transmit and receive in stop modes.

The USART is a very flexible serial interface that supports:

- Asynchronous UART communication,
- SPI (serial peripheral interface) master and slave modes,
- SmartCard ISO 7816 communication
- irDA Serial infra red communication
- LIN (local interconnect network) mode.

It also provides certain features that are useful when implementing Modbus communications.

Applications making use of the USART benefit from the easy and inexpensive connection between devices, which only requires a few pins.

In addition, the USART peripheral is functional in low-power modes. It comes with Transmit and Receive FIFOs and can transmit and receive in stop modes.

- Fully programmable serial interface
 - Data can be 7, 8 or 9 bits
 - Even, odd and no-parity
 - 0.5, 1, 1.5 and 2 stop bits
 - Programmable data order with MSB or LSB first
 - Programmable baudrate generator
 - Configurable oversampling method by 16 or by 8
- Two internal FIFOs for transmit and receive data
- Supports RS-232 and RS-485 hardware flow control
- Dual clock domain allowing:
 - UART functionality and wakeup from low-power mode
 - Baud-rate programming independent of PCLK changes



The USART is a fully programmable serial interface featuring the following configurable parameters:

- data length,
- parity,
- number of stop bits,
- data order,
- baud rate generator
- and a configurable oversampling mode by 8 or by 16.

The USART can operate in FIFO mode and it comes with two FIFOs: Transmit and Receive FIFOs.

You also have the option to use basic RS-232 flow control with CTS (Clear To Send) and RTS (Request To Send) signals.

The RS-485 DE (Driver Enable) signal is also supported. The USART supports a dual clock domain allowing

wakeup from Stop mode and baud rate programming independent of the peripheral clock (PCLK).

This also allows the peripheral clock to be throttled along with the core clock without disrupting communications.

Key Features Continued 4

- Multi-processor communication
- Single-wire half-duplex communication
- Auto baud rate detection
- Receiver timeout feature
- Supports also
 - LIN mode
 - Synchronous mode (Master mode)
 - IrDA SIR encoder encoder
 - Smartcard (ISO/IEC 7816 T=0 and T=1 protocols)
 - Basics to implement Modbus/RTU and Modbus/ASCII

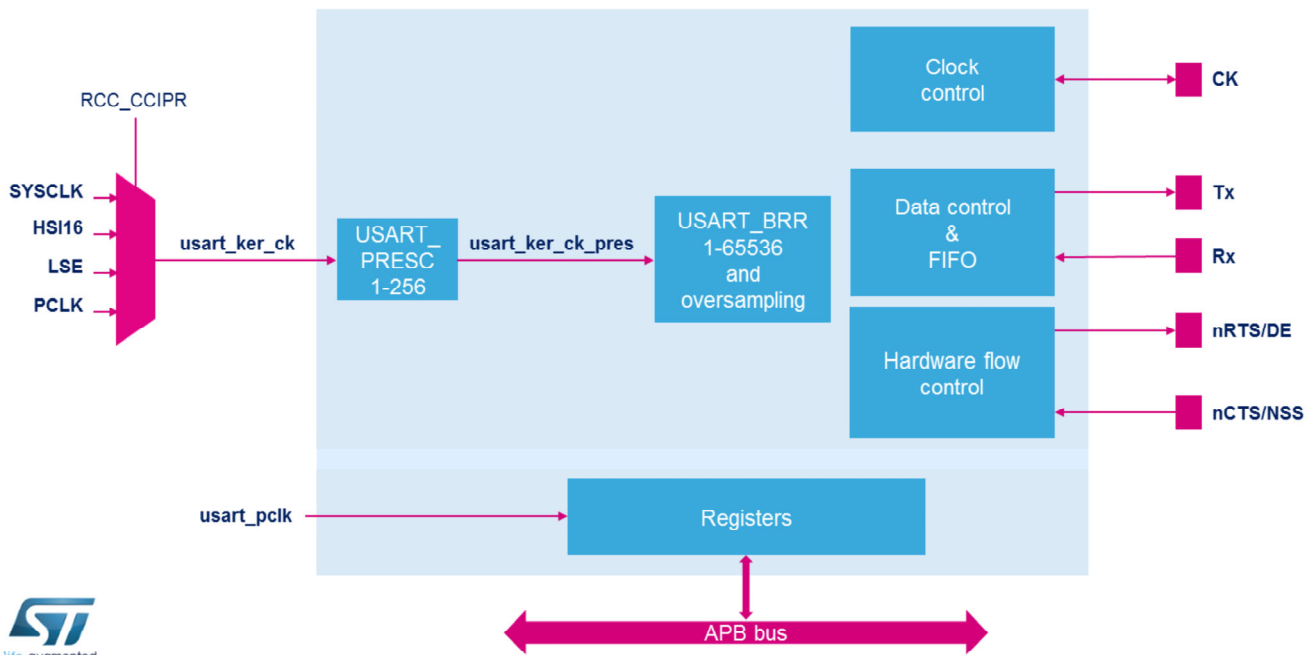


The USART features a multi-processor mode which allows the USART to remain idle when it is not addressed.

In addition to full-duplex communication, single-wire half-duplex mode is also supported.

The USART also offers many other features including auto baud rate detection, receiver timeout and supports several modes which will be described later in the presentation.

Block diagram 5



This is the USART block diagram.

The USART clock source (`usart_ker_ck`) can be selected from several sources: peripheral clock (APB clock PCK), SYSCLK, High Speed Internal 16-MHz oscillator (HSI16), Low Speed External Oscillator (LSE). The USART clock source is divided by a programmable factor in the `USART_PRESC` register in range 1 to 256. Tx and Rx pins are used for data transmission and reception.

`nCTS` and `nRTS` pins are used for RS-232 hardware flow control.

The Driver Enable pin (DE) which is available on the same I/O as `nRTS` is used in RS-485 mode.

The clock output (CK) is dual purpose:

- When the USART is used in Synchronous Master/Slave mode, the clock provided to the slave device is output/input on the CK pin.

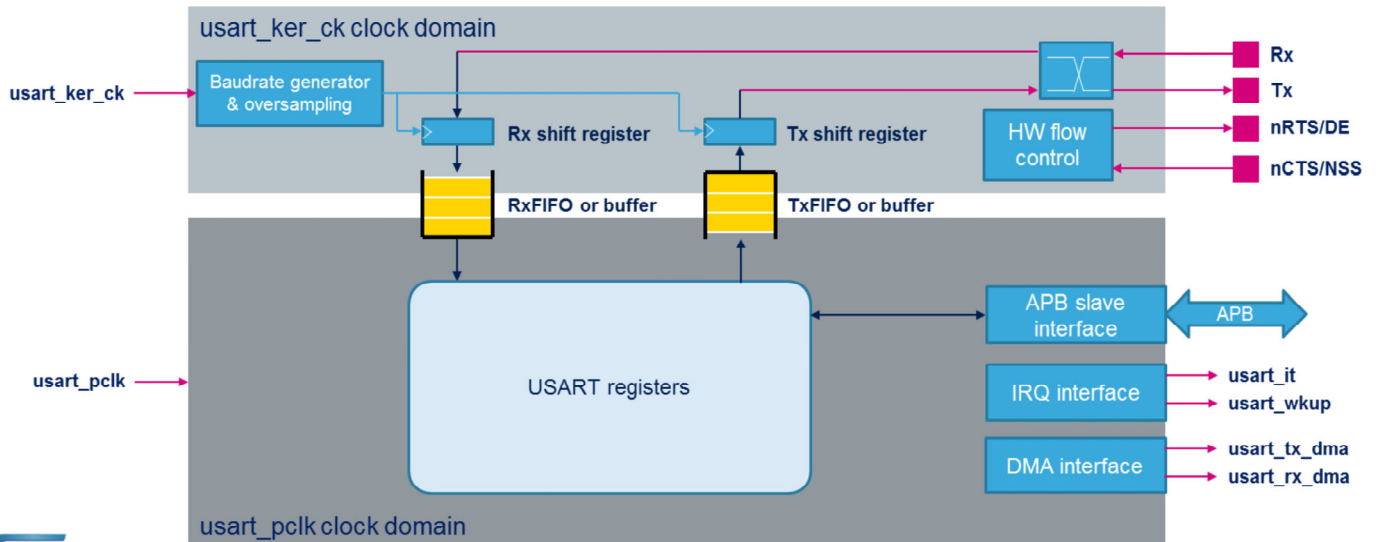
➤ When the USART is used in Smartcard mode, the clock provided to the card is output on the CK pin.

Note that the NSS and nCTS signals share the same pin:

➤ NSS is the slave selection input applied to the device in synchronous slave mode.

Dual clock domain 6

Baudrate programming independent from PCLK reprogramming



The USART has a flexible clocking scheme. The registers are accessed through the APB bus, and the kernel is clocked with `usart_ker_ck` (prescaled or not) which is independent from the APB clock. In order to pass data from one clock domain to the other one, either 8-data FIFOs are used or single data buffers. The USART block is an APB slave that can rely on DMA requests to transfer data to/from memory buffers. The functions of the TX and RX pins can be swapped. This allows to work in the case of a cross-wired connection to another UART.

Different user configurable oversampling techniques

- Oversampling choice affects speed and framing tolerance:

	Oversampling by 8	Oversampling by 16
Benefits	Achieve the maximum speed $\text{usart_ker_ck}/8$	Maximum receiver tolerance to clock deviation is increased.
Drawbacks	Maximum receiver tolerance to clock deviation is reduced.	Maximum speed is limited to $\text{usart_ker_ck}/16$.

- Maximum baud rate depends on selected clock and oversampling:
 - It is 8 Mbaud when clock source is at 64 MHz and Oversampling by 8 is configured.



The USART receiver implements different user-configurable oversampling techniques for data recovery by discriminating between valid incoming data and noise. This allows a trade-off between the maximum communication speed and noise/clock inaccuracy immunity.

Select oversampling by 8 to achieve higher speed (up to $\text{usart_ker_ck_pres}/8$) where usart_ker_ck_pres is the USART clock source frequency. In this case the maximum receiver tolerance to clock deviation is reduced.

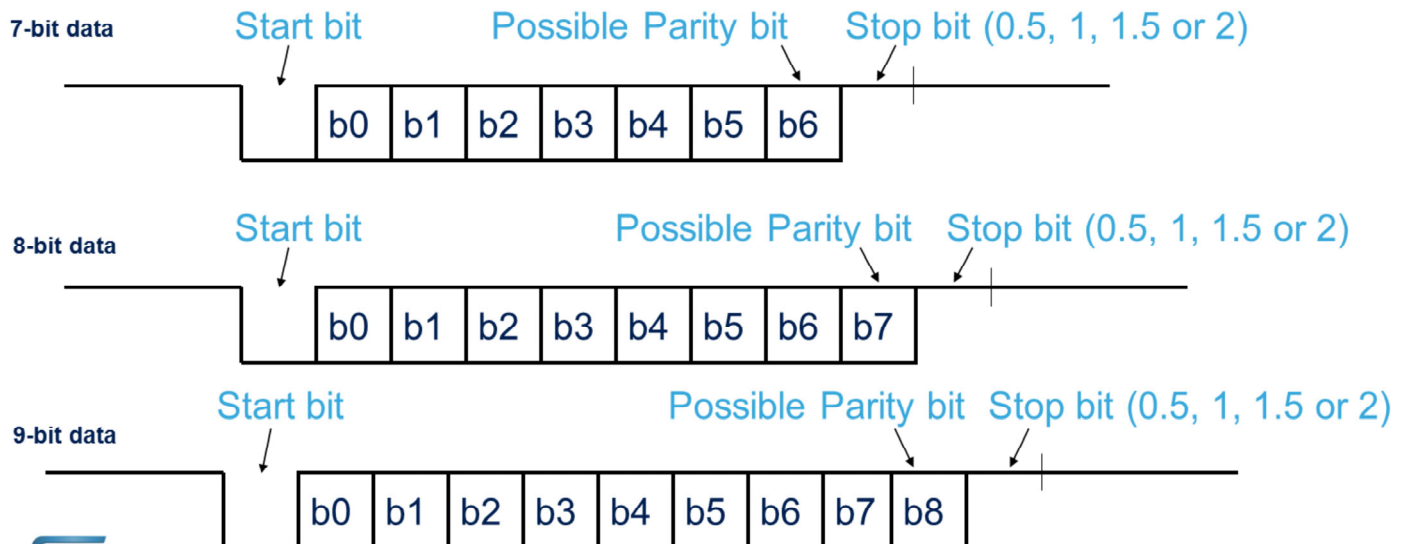
Select Oversampling by 16 ($\text{OVER8} = 0$) to increase the tolerance of the receiver to clock deviations. In this case, the maximum speed is limited to $\text{usart_ker_ck_pres}/16$. The maximum baud rate that can be reached is 8 Mbaud when the clock source is at 64 MHz and Oversampling by 8 is configured.

With other clock sources, and/or higher oversampling ratio, the maximum speed is limited.

Data format – Asynchronous mode

8

Supported data lengths: 7, 8 and 9 bits



The frame format used in Asynchronous mode consists of a set of data bits in addition to bits for synchronization and optionally a parity bit for error checking.

The USART supports 7-, 8- or 9-bit data lengths.

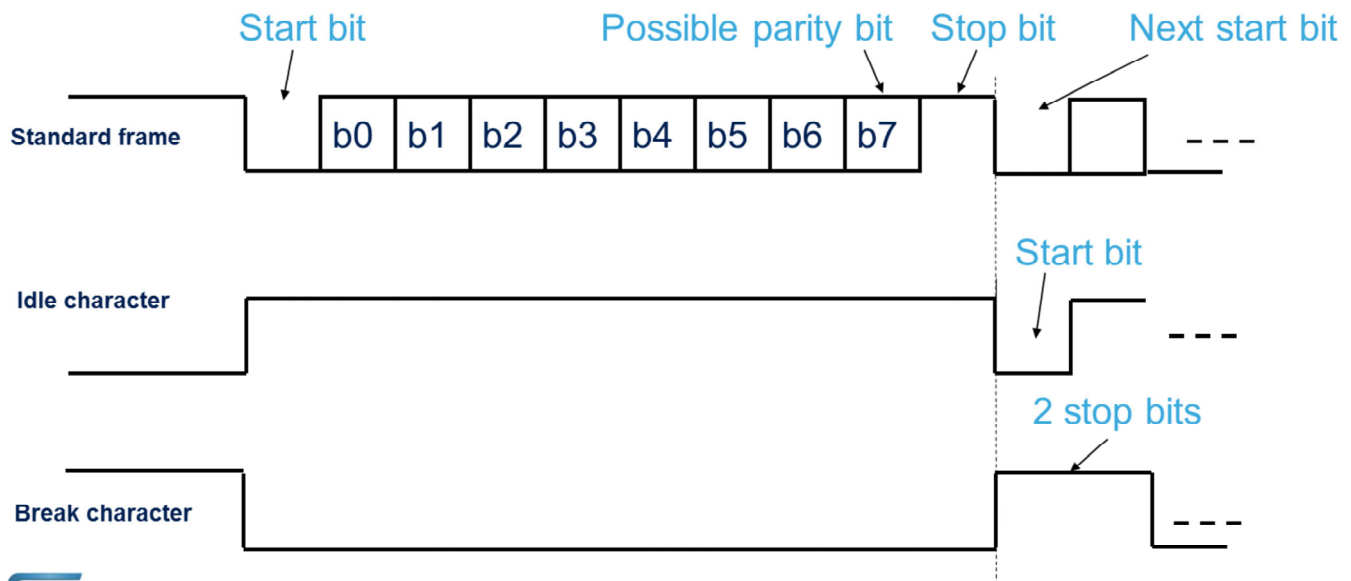
A frame starts with one start-bit, where the line is driven low for a one-bit period. This signals the start of a frame, and is used for synchronization.

The start bit is followed by 7, 8 or 9 data bits. If Parity Control is enabled, the parity bit is transmitted as the last data bit and is included in the data length count.

Finally, a number of stop-bits (0, 1, 1.5 or 2), where the line is driven high, ends the frame.

Data order is programmable with MSB-first or LSB-first shifting.

Idle/Break character 9



The standard frame was described in the previous slide. This slide shows an example of 8-bit data frames configured with 1 stop bit.

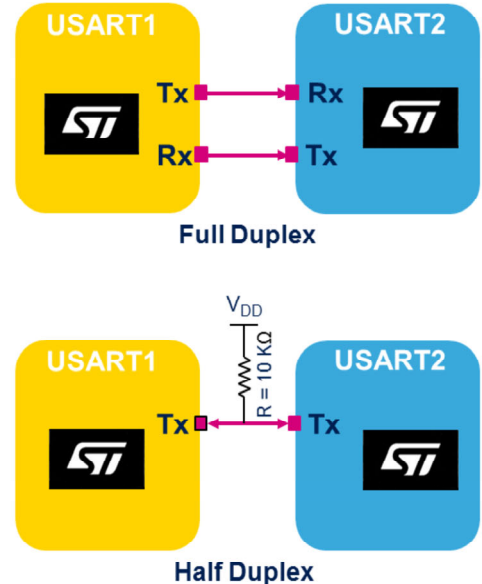
An Idle character is interpreted as an entire frame of “1”s. (The number of “1”s will include the number of stop bits).

A Break character is interpreted on receiving “0”s for a frame period. At the end of the break frame, 2 stop bits are inserted.

Full/Half duplex modes 10

Full-duplex: Two wires
Half-duplex: Single wire

- USART full-duplex communication:
 - Tx and Rx lines are respectively connected with the other interface's Rx and Tx lines.
- USART single-wire half-duplex protocol
 - Tx and Rx lines are internally connected.
 - Tx pin is used for both transmission and reception.



The USART supports full-duplex communication where Tx and Rx lines are respectively connected with the other interface's Rx and Tx lines.

The USART can be configured to follow a single-wire half-duplex protocol where the Tx and Rx lines are internally connected.

In this communication mode, only the Tx pin is used for both transmission and reception.

The Tx pin is always released when no data is transmitted, thus, it acts as a standard I/O in idle or reception modes.

This means that the I/O must be configured so that the Tx pin is configured as an alternate function open-drain with an external pull-up.

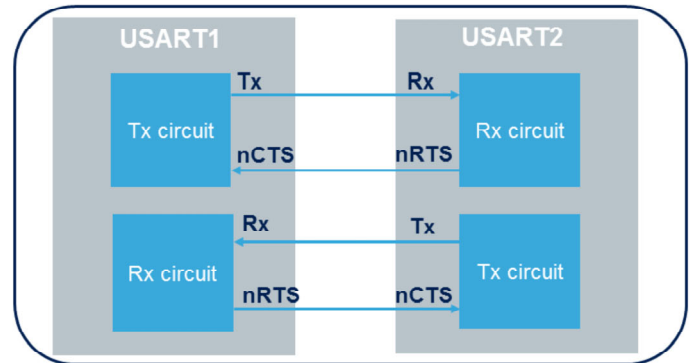
R232 hardware flow control

11

Hardware handshaking to avoid data underrun/overrun

- RS-232 hardware flow control

- RTS (Request To Send) output asserted means that the receiver is ready to accept data.
- CTS (Clear To Send) input asserted means that transmitter can continue communication.
- Particularly useful for half-duplex systems.



In RS-232 communication, it is possible to control the serial data flow between 2 devices by using the nCTS input and the nRTS output.

These two lines allow the receiver and the transmitter to alert each other of their state. The following figure shows how to connect 2 devices in this mode. The idea is to prevent dropped bytes or conflicts in case of half-duplex communication. Both signals are active low.

RS-485 hardware flow control 12

RS485 Hardware control mode

- In RS485 Hardware control mode, the master needs to generate a direction signal to control the transceiver (Physical Layer=PHY)
 - This signal informs the PHY if it must act in send or receive mode.
 - It uses the DE (Driver Enable) pin to activate the external RS-485 bus driver.
- The DE and nRTS signals are available on the same pin.



For serial half-duplex communication protocols like RS-485, the master needs to generate a direction signal to control the transceiver (Physical Layer). This signal informs the Physical Layer if it must act in send or receive mode.

In RS-485 mode, a control line is used: the Driver Enable pin is used to activate the external transceiver control. DE shares the pin with nRTS.

Multi-processor communication 13

Communication between several devices

- In multi-processor communication, it is desirable that only the intended message recipient should actively receive the message.
- The devices not being addressed are put into Mute mode.
- Mute mode can be controlled using two methods:
 - Idle line detection
 - Address mark detection



To simplify communication between multiple processors, the USART supports a multi-processor mode.

In multi-processor communication, it is desirable that only the intended message recipient should actively receive the message.

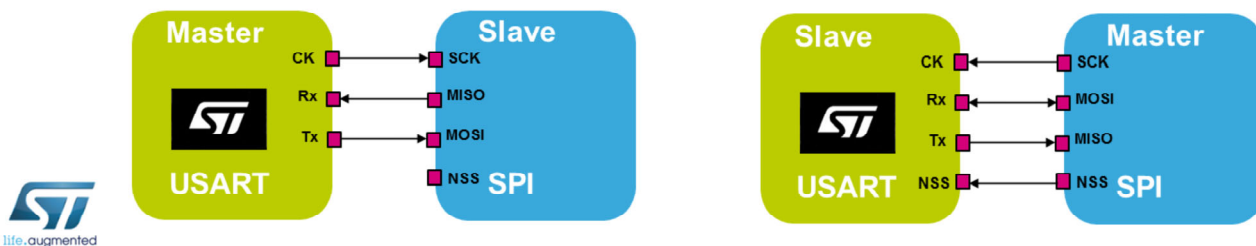
The devices not being addressed are put into Mute mode.

The USART can enter or exit from Mute mode using one of two methods:

- Idle line detection
- Address mark detection.

USART used as SPI Master/Slave

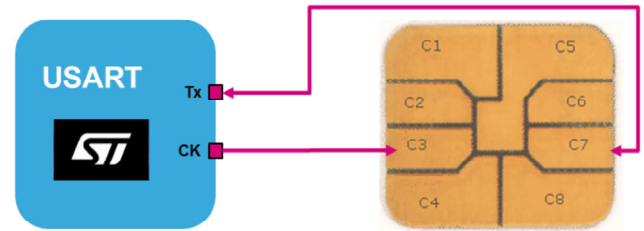
- Full-duplex or simplex synchronous communication mode:
 - SPI master/slave modes
 - Programmable clock polarity (CPOL) and phase (CPHA)
 - Programmable data order with MSB or LSB first
 - Clock output/input on CK pin
 - No clock pulses during the Start and Stop bits
 - Transmit underrun error (Only in SPI slave mode)
 - NSS management (Software or hardware management) (Only in SPI slave mode)



The USART can also communicate synchronously. It can operate as a SPI in Master or Slave mode with programmable clock polarity (CPOL) and phase (CPHA) and programmable data order with MSB or LSB first. The clock is output (in case of Master mode) or input (in case of Slave mode) on the CK pin. No clock pulses are provided during the start and stop bits. When the USART is configured in SPI slave mode, it supports the Transmit underrun error and the NSS hardware or software management.

USART interface for Smartcards and Security Access Module

- Half-duplex mode
- Clock provided to Smartcard on CK pin
- Programmable clock prescaler to guarantee a wide range of clock inputs
- ISO/IEC 7816 T=0 and T=1 protocols supported
- Both direct and inverse convention are available



The USART can be used in Smartcard mode, based on a half-duplex communication.

The clock is output to the Smartcard on the CK pin.

It supports the T=0 protocol and provides many features allowing support for T=1.

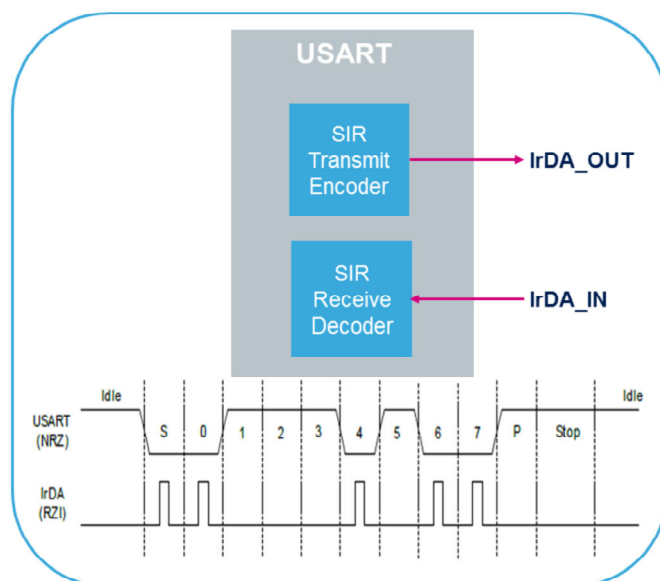
Both direct and inverse conventions are supported directly by hardware.

IrDA SIR Encoder Decoder

16

USART interface for infrared wireless connection

- Half-duplex communication
- The data from and to the USART is represented in a NRZ (Non Return to Zero) format
- For IrDA, the required format is RZI (Return to Zero Inverted)
- The SIR Tx Encoder modulates the signal before it leaves the USART
 - Similarly, the input signal is demodulated in the SIR Rx Decoder
- Max bit rate is 115.2 Kbits/s
- The pulse width is 3/16 bit duration in normal mode



The USART supports IrDA specifications which is a half-duplex communication protocol.

The data from and to the USART is represented in a NRZ (Non Return to Zero) format where the signal value is at the same level through the entire bit period.

For IrDA, the required format is RZI (Return to Zero Inverted) where a “1” is signaled by holding the line low, and a “0” is signaled by a short high pulse.

The SIR Transmit encoder modulates the Non Return to Zero (NRZ) transmit bit stream output from the USART.

The SIR receive decoder demodulates the return-to-zero bit stream from the infrared detector and outputs the received NRZ serial bit stream to the USART.

The USART only supports bit rates up to 115.2 Kbits/s for the SIR ENDEC.

In normal mode, the transmitted pulse width is specified

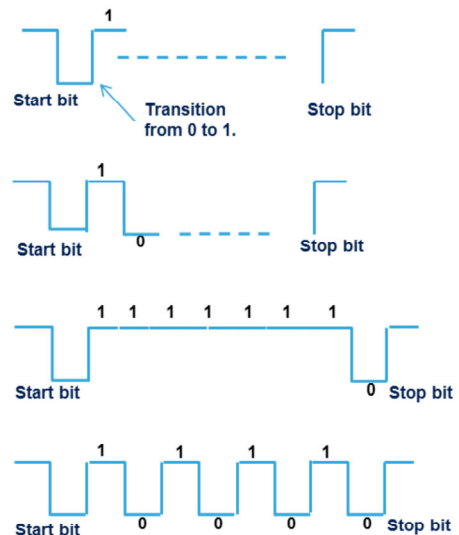
as $3/16$ of a bit period.

Auto baudrate detection

17

Automatic baudrate configuration - UART receiver

- The USART is able to automatically determine the baud rate based on the reception of one character
- The received character can be:
 - Any character starting with a bit at '1'
 - Any character starting with a 10xx pattern
 - 0x7F
 - 0x55



The USART receiver is able to detect and automatically configure the baud rate based on the reception of one character.

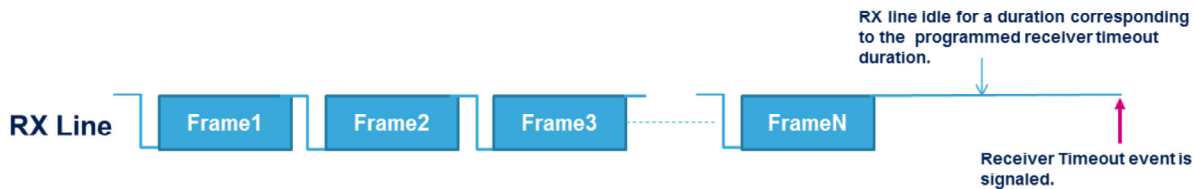
The received character can be:

- Any character starting with a bit at 1. In this case, the USART measures the duration of the start bit (from falling edge to rising edge).
- Any character starting with a 10xx pattern. In this case, the USART measures the duration of the start and of the 1st data bit. The duration is measured from falling edge to falling edge, ensuring better accuracy in the case of slow signal slopes.
- 0x7F character frame. In this case, the baud rate is updated first at the end of the start bit then at the end of bit 6.
- A 0x55 character frame. In this case, the baud rate is updated first at the end of the start bit, then at the end

of bit 0 bit and finally at the end of bit 6. In parallel, another check is performed for each intermediate transition of the RX line.

Receiver timeout 18

- When the USART receiver does not receive new data for a programmed amount of time, this can be signaled to the application via a receiver timeout event



- The USART receiver timeout counter starts counting:
 - From the end of the first stop bit in case of 1 and 1.5 stop bit configuration
 - From the end of the second stop bit in case of 2 stop bits configuration
 - From the beginning of the stop bit in case of 0.5 stop bit configuration



The USART supports a receiver timeout feature. When the USART doesn't receive new data for a programmed amount of time, a receiver timeout event is signaled and an interrupt is generated if enabled.

The USART receiver timeout counter starts counting:

- From the end of the first stop bit in case of 1 and 1.5 stop bit configuration.
- From the end of the second stop bit in case of 2 stop bits configuration.
- From the beginning of the stop bit in case of 0.5 stop bit configuration.

Transmission and reception FIFOs

19

Transmission/Reception even during stop modes

- FIFO mode is enabled/disabled by software
- Transmit FIFO (TXFIFO) and Receive FIFO (RXFIFO)
- TXFIFO and RXFIFO size is 8 data
- FIFO mode is not available in IrDA and LIN modes
- FIFOs are in kernel clock domain → ability to receive or transmit even in Stop mode
- Adjustable TXFIFO and RXFIFO thresholds to assert interrupt requests



The USART can operate in FIFO mode which is enabled/disabled by software. It is disabled by default. The USART comes with a Transmit FIFO (TXFIFO) and a Receive FIFO (RXFIFO), each being 8 data deep. When the IrDA and LIN modes are used, the FIFO mode is not supported.

Provided that the TXFIFO and RXFIFO are clocked by the kernel clock, it is possible to transmit and receive data even in Stop mode.

It is possible to configure TXFIFO and RXFIFO thresholds, used mainly to avoid underrun/overrun issue while waking up from Stop mode.

Wake up from Stop mode 20

- The USART is able to wake up the MCU from Stop mode when the USART clock source is:
 - HSI
 - LSE
- The sources of wakeup can be:
 - A specific wakeup event triggered by:
 - Start bit
 - Address match
 - Any received data
 - Standard RXNE interrupt when FIFO management is disabled
 - FIFO event interrupts when FIFO management is enabled: RXFIFO full, TXFIFO empty, RXFIFO/TXFIFO reach the programmed threshold



The USART is able to wake up the MCU from Stop mode when the USART clock source is the HSI or LSE clock.

The sources of wakeup can be:

- A specific wakeup event which is triggered by either a start bit or an address match or any received data.
- An RXNE interrupt when FIFO management is disabled or
- FIFO event interrupts when FIFO management is enabled:
 - Receive FIFO full interrupt
 - Transmit FIFO empty interrupt
 - Receive FIFO threshold interrupt
 - Transmit FIFO threshold interrupt

Interrupt event	Description	Wakeup request ?
Transmit data register empty	Set when the Transmit Data register is empty	NO
Transmit complete	Set when the data transmission is complete and both data and shift registers are empty	NO
CTS	Set when the nCTS input toggles	NO
Receive data register not empty	Set when the Receive Data register contains data	YES
Idle line	Set when an idle line is detected	NO
Character match	Set when the received data corresponds to the programmed address	NO
Receiver timeout	Set when there is no activity on the Rx line for a duration equal to the programmed timeout	NO
Transmission Complete Before Guard Time	This flag is set just after the end of frame transmission and if no NACK has been received from the card (ISO/IEC 7816 mode)	NO



This slide and the two next ones provide the list of interrupt events, detailing their cause and indicating whether these events can be used as wakeup requests. Several events can provide an interrupt:

- The Transmit Data Register Empty flag is set when the Transmit Data register is empty and ready to be written.
- The Transmit Complete flag is set when the data transmission is complete and both data and shift registers are empty.
- The CTS flag is set when the nCTS input toggles.
- The Receive Data Register Not Empty flag is set when the Receive Data register contains data ready to be read.
- The Idle Line flag is set when an idle line is detected.
- The Character Match flag is set when the received data corresponds to the programmed address.

- The Receiver Timeout flag is set when there is no activity on the Rx line for a programmed duration.
- The Transmission Complete Before Guard Time flag is set after the end of frame transmission and if no NACK has been received from the card.

Interrupt event	Description	Wakeup request ?
LIN break	Set when a LIN break frame is detected	NO
End of block	Set when a complete block was received	NO
Wakeup from stop mode	Set when the wakeup event is verified	YES
Transmit FIFO not full	Set when the Transmit FIFO is not full	NO
Transmit FIFO empty	Set when the Transmit FIFO is empty	YES
Transmit FIFO threshold reached	Set when programmed threshold is reached	YES
Receive FIFO not empty	Set when the Receive FIFO is not empty	YES
Receive FIFO full	Set when the Receive FIFO is full	YES
Receive FIFO threshold reached	Set when the programmed threshold is reached	YES



The End of Block flag is set when a complete block is received.

The Wakeup from Stop Mode flag is set when the wakeup event is verified.

The LIN break flag is set when a LIN break frame is detected.

The Transmit FIFO not full flag is set when the Transmit FIFO is not full.

The Transmit FIFO empty flag is set when the Transmit FIFO is empty.

The Transmit FIFO threshold flag is set when programmed threshold is reached.

The Receive FIFO not empty flag is set when the Receive FIFO is not empty.

The Receive FIFO full flag is set when the Receive FIFO is full.

The Receive FIFO threshold flag is Set when the

programmed threshold is reached.

Interrupt event	Description	Wakeup request ?
Overrun error	Set when an overrun error occurs.	NO
Parity error	Set when a parity error occurs.	NO
Framing error	Set when a framing error occurs.	NO
Noise error	Set when a noise is detected on the received frame.	NO
Auto-baudrate error	Set when the baudrate measurement failed.	NO
Underrun error	Set when an underrun error occurs in synchronous slave mode.	NO

Several errors flags can be generated:

- The Overrun error flag is set when an overrun error occurs.
- The Parity error flag is set when a parity error occurs.
- The Framing error flag is set when a framing error occurs.
- The Noise error flag is set when a noise is detected on the received frame.
- The Auto-baudrate error flag is set when the baudrate measurement failed.
- The Underrun error flag is set when an underrun error occurs in synchronous slave mode.

- The USART is capable of performing continuous communications using the DMA.
- The DMA requests for Rx buffer and Tx buffer are generated independentlyDMA requests are triggered by:
 - Transmit data register empty and Receive data register full when FIFO management is disabled.
 - Transmit FIFO not full and Receive FIFO not empty when FIFO management is enabled.

The DMA requests can be generated when Receive Buffer Not Empty or Transmit Buffer Empty flags are set when FIFO management is disabled.

The DMA requests can also be generated when the Transmit FIFO not full and Receive FIFO not empty flags are set when FIFO management is enabled.

Low-power modes 25

Mode	Description
Run Low power Run	Active
Sleep Low power Sleep	Active Peripheral interrupts cause the device to exit Sleep mode.
Stop 0 Stop 1	The content of the USART registers is kept The USART is able to wake up the MCU from Stop mode when the USART clock is set to HSI or LSE
Standby Shutdown	Powered-down The peripheral must be reinitialized after exiting Standby or Shutdown mode



The USART peripheral is active in Run, Sleep and low-power modes. The USART interrupts cause the device to exit Sleep and Low-power Sleep modes.

The USART is able to wake up the MCU from Stop 0 and Stop 1 modes when the USART clock is set to HSI or LSE.

In Standby and shutdown modes, the peripheral is in power-down, and it must be reinitialized after exiting Standby or Shutdown mode.

STM32G0 USART/LPUART features

26

USART features	USART1/2	USART3/4	LPUART
Hardware flow control for modem	X	X	X
Multiprocessor communication	X	X	X
Synchronous mode (Slave/Master)	X	X	-
Smartcard mode	X	-	-
Single wire half duplex communication	X	X	X
IrDA SIR ENDEC	X	-	-
LIN mode	X	-	-
Dual clock domain and wakeup from Stop mode	X	-	X
Receiver timeout	X	-	-
Auto baudrate detection	X	-	-
Driver enable	X	X	X
Data length	7, 8 and 9 bits		
TX/RX FIFO	X	-	X
TX/RX FIFO size (bytes)	8	-	8



This table also highlights the differences between the 4 USARTs and the LPUART.

The LPUART does not support:

- Synchronous mode
- IrDA communication
- LIN mode

STM32F0 USART vs. STM32G0 USART

27

- STM32G0 USART has same specification as STM32F0 USART, with some added features:
 - Clock prescaler
 - SPI slave mode
 - 2 FIFOs: RXFIFO and TXFIFO



This slides shows the extended features of the STM32G0 vs the STM32F0.

- Refer to these peripherals trainings linked to this peripheral
 - GPIO (Alternate function configuration)
 - Reset and clock controller (RCC)
 - Power controller (PWR)
 - Interrupts (NVIC and EXTI)
 - Direct memory access controller (DMA)



This is a list of peripherals related to the USART. Please refer to these trainings for more information if needed.

- General-purpose input/outputs
- Reset and clock controller
- Power controller
- Interrupts controller
- Direct memory access controller